

Laporan Praktikum Kontrol Cerdas

Nama : Muhammad Daffa Adyatma P

NIM : 224308038

Kelas : TKA-6B

Akun Github (Tautan) : <https://github.com/dafferdogan>

Student Lab Assistant :Mbak Dwi

1. Judul Percobaan : ***Human Pose Estimation***

2. Tujuan Percobaan :

- Mempelajari dasar-dasar Human Pose Estimation
- Melakukan inferensi pose pada gambar, video, dan kamera real-time.
- Melatih dan menguji menggunakan klasifikasi SVM
- Menggunakan GitHub untuk version control dan dokumentasi praktikum.

3. Landasan Teori :

Machine Learning merupakan sistem yang mampu belajar sendiri peran manusia dalam pengambilan keputusan dengan mengambil keputusan secara otomatis, belajar dari data, dan menentukan pola tanpa harus berulang kali diprogram oleh manusia (Prasetyo & Dewayanto., 2024). Untuk bisa menggunakan *machine learning* harus memiliki data. Data yang digunakan dibagi menjadi dua data training dan data testing. Data training digunakan untuk melatih algoritma, sedangkan data testing digunakan untuk mengetahui performa yang telah dilatih sebelumnya ketika menemukan data baru yang belum pernah diberikan dalam data training (Yudianto & Fatta, 2020). Machine Learning memiliki beberapa jenis, seperti, supervised learning, unsupervised learning dan reinforcement learning.

Deep learning merupakan pembelajaran mesin yang berkaitan dengan algoritma dimana cara kerja dari algoritma ini meniru struktur dan fungsi otak yang disebut jaringan saraf tiruan (Rochmawati et al., 2021). *Deep learning* sangat berguna ketika mencoba untuk memahami pola dari data yang tidak terstruktur.

Jaringan saraf kompleks dalam deep learning dirancang untuk meniru cara kerja otak manusia sehingga komputer dapat dilatih untuk menangani abstraksi dan masalah yang kurang terdefinisi dengan baik (Windiawan & Suharso, 2021). metode dalam klasifikasi dalam Super Vector Machine. Super Vector Machine berfungsi untuk memperoleh hasil prediksi pengujian, yang mana hasil dari prediksi untuk pengujian diperoleh dari kelompok yang berupa feature vector. Support Vector Machine (SVM) adalah teknik seleksi yang menghasilkan hasil dengan tingkat akurasi klasifikasi tertinggi dengan membandingkan sekumpulan parameter standar dengan nilai diskrit yang disebut sebagai himpunan kandidat (Septhya et al., 2023).

Human Pose Estimation dapat didefinisikan sebagai pengaturan sendi manusia dengan cara tertentu. Oleh karena itu masalah Human Pose Estimation dapat didefinisikan sebagai lokalisasi sendi manusia atau landmark yang telah ditentukan. Dalam gambar dan video, ada beberapa jenis estimasi pose, termasuk tubuh, wajah, dan tangan khususnya dalam computer vision. Estimasi pose manusia dari video memainkan peran penting dalam berbagai aplikasi seperti mengukur latihan fisik, pengenalan bahasa isyarat, dan kontrol gerakan seluruh tubuh (Abdul Muthalib et al., 2023).

4. Analisis dan Diskusi :

-Analisis

Human Pose Estimation (HPE) menggunakan Support Vector Machine (SVM) dalam percobaan ini memiliki berbagai kegunaan yang signifikan dalam analisis gerakan tangan. Salah satu kegunaan utama dari HPE dengan SVM adalah untuk klasifikasi gerakan tangan berdasarkan fitur yang diekstrak dari posisi landmark tangan yang terdeteksi. Dengan menggunakan model SVM, sistem dapat mengidentifikasi berbagai gerakan tangan. Dalam percobaan ini, MediaPipe digunakan sebagai alat untuk mendeteksi landmark tangan secara real-time. MediaPipe menyediakan algoritma yang efisien dan akurat untuk mengekstrak posisi landmark tangan, yang kemudian digunakan sebagai input untuk model SVM.

Dengan demikian, kombinasi MediaPipe dan SVM memungkinkan sistem untuk melakukan deteksi dan klasifikasi gerakan tangan dengan cepat dan efektif. Dalam percobaan ini juga menganalisis Gerakan tangan. Dengan mendeteksi dan mengklasifikasikan gerakan tangan secara real-time berdasarkan data dari dataset, sistem dapat memberikan respons yang lebih intuitif dan interaktif. Dataset keypoint untuk menentukan posisi tangan digunakan sebagai data dalam analisis dan pengenalan gerakan tangan. Dataset ini biasanya terdiri dari koordinat landmark yang merepresentasikan posisi berbagai titik pada tangan, seperti jari, pergelangan, dan bagian lainnya. Dengan menggunakan dataset ini, sistem dapat melakukan beberapa hal penting. dataset keypoint menggunakan pelatihan model machine learning, seperti Support Vector Machine (SVM), untuk mengenali dan mengklasifikasikan berbagai pose tangan. Setiap pose tangan dapat diwakili oleh sekumpulan koordinat yang menunjukkan posisi relatif dari landmark tangan. Dengan data ini, model dapat belajar untuk membedakan antara berbagai gerakan, seperti mengangkat tangan, menggenggam, atau melakukan gestur tertentu. Dataset ini juga berfungsi untuk ekstraksi fitur yang relevan. Dari koordinat landmark, kita dapat menghitung berbagai fitur, seperti jarak antar landmark, sudut antara sendi, dan pola gerakan. Fitur-fitur ini sangat penting untuk meningkatkan akurasi klasifikasi gerakan tangan.

-Diskusi

Penerapan dataset keypoint untuk menentukan posisi tangan, baik tangan kanan maupun tangan kiri, menyoroti betapa pentingnya data dalam pengembangan sistem Human Pose Estimation (HPE) yang efektif. Dataset ini berfungsi tidak hanya sebagai sumber informasi untuk melatih model machine learning, seperti Support Vector Machine (SVM), tetapi juga sebagai dasar untuk ekstraksi fitur yang relevan. Dengan adanya koordinat landmark yang menggambarkan posisi berbagai titik pada tangan, model dapat belajar untuk mengenali dan mengklasifikasikan berbagai pose tangan dengan lebih akurat, baik untuk tangan kanan maupun tangan kiri. Dengan menghitung fitur-fitur seperti jarak antar landmark dan sudut antara sendi, sistem dapat lebih efektif dalam membedakan antara gerakan yang serupa, seperti

mengangkat tangan kanan dan tangan kiri. Evaluasi kinerja model juga merupakan aspek penting dalam diskusi ini. Setelah model dilatih menggunakan dataset keypoint, penting untuk menguji kemampuannya dalam mengenali pose tangan k untuk tangan kanan maupun tangan kiri

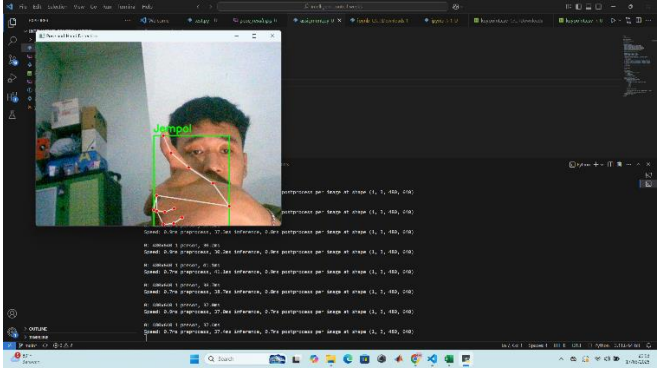
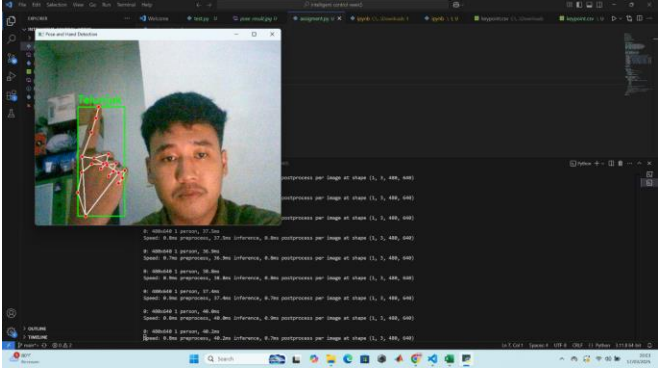
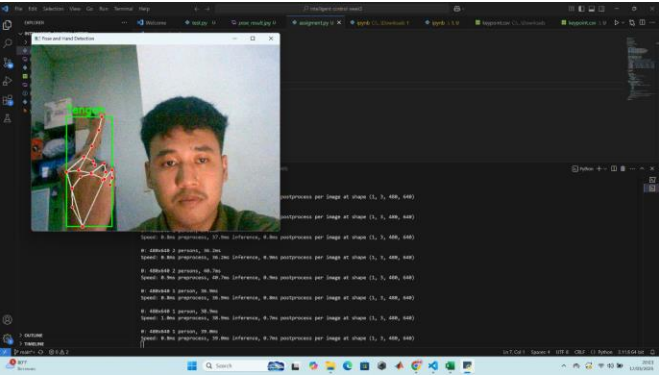
5. Assignment :

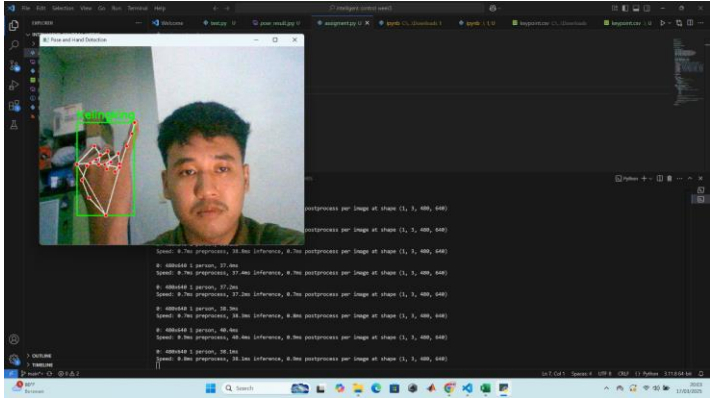
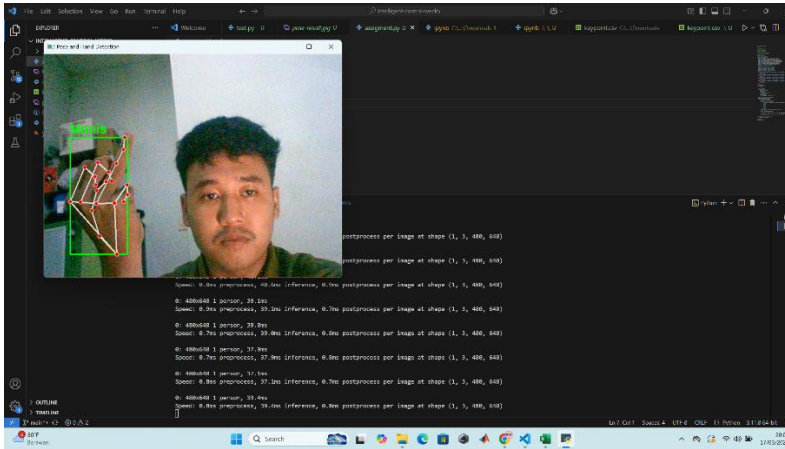
kode memuat dataset dengan menggunakan `pd.read_csv(file_path)`, di mana `file_path` adalah lokasi file CSV yang berisi data keypoint. Setelah dataset dimuat ke dalam DataFrame `df`, langkah selanjutnya adalah mengganti nama kolom untuk memastikan konsistensi dalam jumlah fitur. Kode ini menggunakan `df.shape[1]` untuk mendapatkan jumlah kolom dalam DataFrame dan kemudian mengganti nama kolom menjadi `feature_0`, `feature_1`, dan seterusnya, hingga jumlah kolom yang ada. Ini dilakukan dengan list comprehension `[f"feature_{i}" for i in range(num_features)]`. Selanjutnya, kode mendefinisikan `expected_features` sebagai 42, yang merupakan jumlah total landmark (21) dikalikan dengan 2 (untuk koordinat x dan y), dengan asumsi bahwa data z tidak tersedia. Kode kemudian menghitung `actual_features` dengan mengurangi 1 dari jumlah kolom, karena kolom pertama berisi label. Setelah itu, kode memeriksa apakah jumlah fitur yang sebenarnya (`actual_features`) kurang dari jumlah fitur yang diharapkan (`expected_features`). Jika jumlah fitur yang ada kurang dari yang diharapkan, kode akan mengeluarkan pengecualian `ValueError` dengan pesan yang menjelaskan perbedaan jumlah fitur. Jika jumlah fitur sesuai, kode melanjutkan untuk mengekstrak fitur dari DataFrame dengan menggunakan `df.iloc[:, 1:expected_features+1]`, yang mengambil semua kolom dari indeks 1 hingga `expected_features` untuk membentuk variabel `X`, yang berisi data fitur. Terakhir, kode menetapkan variabel `y` untuk menyimpan label target dengan mengambil kolom pertama dari DataFrame menggunakan `df.iloc[:, 0]`. Selanjutnya, mengimpor modul `mp_hands` dari `mp.solutions.hands`, yang menyediakan fungsi dan kelas yang diperlukan untuk mendeteksi tangan dalam gambar. Selanjutnya, `mp_drawing` diimpor dari `mp.solutions.drawing_utils`, yang berfungsi untuk menggambar landmark serta koneksi antara titik-titik landmark

pada gambar. Setelah itu, objek hands diinisialisasi dengan memanggil `mp_hands.Hands()`, di mana parameter `min_detection_confidence` dan `min_tracking_confidence` diatur masing-masing ke 0.5. Parameter ini menentukan ambang batas kepercayaan minimum untuk deteksi dan pelacakan tangan; nilai 0.5 menunjukkan bahwa sistem akan menganggap hasil deteksi dan pelacakan sebagai valid jika tingkat kepercayaan model mencapai 50% atau lebih. Kemudian, mengonversi gambar dari format BGR ke RGB menggunakan fungsi `cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)`, yang sangat penting karena pustaka MediaPipe memproses gambar dalam format RGB, sehingga memastikan bahwa data gambar yang dikirim ke model berada dalam format yang tepat. Setelah konversi, gambar yang telah diproses dikirim ke fungsi `hands.process(image)`, yang bertugas untuk mendeteksi landmark tangan dalam gambar; jika tangan terdeteksi, kode akan memasuki blok `if results.multi_hand_landmarks:`. Di dalam blok ini, setiap landmark tangan dan informasi mengenai tangan (apakah tangan kanan atau kiri) diekstrak menggunakan loop `for`, di mana kode menginisialisasi daftar `keypoints` untuk menyimpan koordinat landmark dan variabel untuk menghitung koordinat bounding box, serta menghitung posisi `x` dan `y` berdasarkan ukuran gambar dan menambahkan nilai-nilai ini ke dalam daftar `keypoints`. Selanjutnya, kode memperbarui koordinat bounding box dengan menggunakan fungsi `min` dan `max` untuk menentukan batas terkecil dan terbesar dari landmark yang terdeteksi, yang memungkinkan visualisasi area di sekitar tangan yang terdeteksi. Kode ini juga menggambar nomor indeks pada setiap sendi tangan menggunakan `cv2.putText`, yang membantu dalam visualisasi, dan setelah itu, landmark tangan digambar menggunakan `mp_drawing.draw_landmarks(frame, hand_landmarks, mp_hands.HAND_CONNECTIONS)`, yang menghubungkan titik-titik landmark dengan garis. Untuk menentukan apakah tangan yang terdeteksi adalah tangan kiri atau kanan, kode memeriksa label `handedness` yang dihasilkan oleh model; jika labelnya "Left", maka tangan yang terdeteksi adalah tangan kanan, dan sebaliknya, informasi ini ditampilkan di layar menggunakan `cv2.putText`. Dengan langkah-

langkah ini, kode tidak hanya mendeteksi landmark tangan tetapi juga memberikan visualisasi yang jelas mengenai posisi dan identitas tangan yang terdeteksi.

6. Data dan Output Hasil :

No	Variabel	Hasil Pengamatan
1.	Mendeteksi jari jempol	 The screenshot shows a person's hand with the thumb extended. A green bounding box is drawn around the thumb, and red lines connect the joints of the thumb. The application window displays the detected hand and the corresponding landmarks.
2.	Mendeteksi jari telunjuk	 The screenshot shows a person's hand with the index finger extended. A green bounding box is drawn around the index finger, and red lines connect the joints of the index finger. The application window displays the detected hand and the corresponding landmarks.
3.	Mendeteksi jari tengah	 The screenshot shows a person's hand with the middle finger extended. A green bounding box is drawn around the middle finger, and red lines connect the joints of the middle finger. The application window displays the detected hand and the corresponding landmarks.

4.	Mendeteks i jari kelinking	 <pre> performance per image at shape (1, 3, 480, 640) 0: 4500640 1 person, 29.1ms Score: 8.79s preprocess, 27.9ms inference, 8.7ms postprocess per image at shape (1, 3, 480, 640) 0: 4500640 1 person, 29.1ms Score: 8.79s preprocess, 27.9ms inference, 8.7ms postprocess per image at shape (1, 3, 480, 640) 0: 4500640 1 person, 29.1ms Score: 8.79s preprocess, 27.9ms inference, 8.7ms postprocess per image at shape (1, 3, 480, 640) 0: 4500640 1 person, 29.1ms Score: 8.79s preprocess, 27.9ms inference, 8.7ms postprocess per image at shape (1, 3, 480, 640) 0: 4500640 1 person, 29.1ms Score: 8.79s preprocess, 27.9ms inference, 8.7ms postprocess per image at shape (1, 3, 480, 640) 0: 4500640 1 person, 29.1ms Score: 8.79s preprocess, 27.9ms inference, 8.7ms postprocess per image at shape (1, 3, 480, 640) 0: 4500640 1 person, 29.1ms Score: 8.79s preprocess, 27.9ms inference, 8.7ms postprocess per image at shape (1, 3, 480, 640) 0: 4500640 1 person, 29.1ms Score: 8.79s preprocess, 27.9ms inference, 8.7ms postprocess per image at shape (1, 3, 480, 640) 0: 4500640 1 person, 29.1ms Score: 8.79s preprocess, 27.9ms inference, 8.7ms postprocess per image at shape (1, 3, 480, 640) </pre>
5.	Mendeteks i jari manis	 <pre> performance per image at shape (1, 3, 480, 640) 0: 4500640 1 person, 29.1ms Score: 8.79s preprocess, 27.9ms inference, 8.7ms postprocess per image at shape (1, 3, 480, 640) 0: 4500640 1 person, 29.1ms Score: 8.79s preprocess, 27.9ms inference, 8.7ms postprocess per image at shape (1, 3, 480, 640) 0: 4500640 1 person, 29.1ms Score: 8.79s preprocess, 27.9ms inference, 8.7ms postprocess per image at shape (1, 3, 480, 640) 0: 4500640 1 person, 29.1ms Score: 8.79s preprocess, 27.9ms inference, 8.7ms postprocess per image at shape (1, 3, 480, 640) 0: 4500640 1 person, 29.1ms Score: 8.79s preprocess, 27.9ms inference, 8.7ms postprocess per image at shape (1, 3, 480, 640) 0: 4500640 1 person, 29.1ms Score: 8.79s preprocess, 27.9ms inference, 8.7ms postprocess per image at shape (1, 3, 480, 640) 0: 4500640 1 person, 29.1ms Score: 8.79s preprocess, 27.9ms inference, 8.7ms postprocess per image at shape (1, 3, 480, 640) 0: 4500640 1 person, 29.1ms Score: 8.79s preprocess, 27.9ms inference, 8.7ms postprocess per image at shape (1, 3, 480, 640) 0: 4500640 1 person, 29.1ms Score: 8.79s preprocess, 27.9ms inference, 8.7ms postprocess per image at shape (1, 3, 480, 640) </pre>

7. Kesimpulan

Berdasarkan praktikum dan analisis yang telah dilakukan, maka dapat diambil kesimpulan yaitu:

- Sistem berhasil mendeteksi tangan secara *real-time* dengan akurasi tinggi dalam pencahayaan optimal.

- Dapat mengenali gestur untuk mendeteksi tangan kanan dan kiri secara efektif.
- MediaPipe digunakan untuk mendeteksi landmark tangan dan menggambarinya pada setiap sendi tangan secara *real-time*.
- SVM digunakan untuk mengklasifikasi posisi gesture tangan kanan maupun tangan kiri.
- Tidak mendeteksi sendi” selain sendi tangan.

8. Saran :

Untuk pengenalan gesture tangan dalam kode deteksi landmark bisa menggunakan algoritma pembelajaran mesin yang lebih kompleks untuk mengklasifikasikan berbagai gesture berdasarkan posisi landmark yang sesuai berdasarkan dengan datasets yang ada.

9. Daftar Pustaka :

Abdul Muthalib, M., Irfan, I., Kartika, K., & Selamat Meliala, S. M. (2023).

PENGIRAAN POSE MODEL MANUSIA PADA REPETISI KEBUGARAN
AI PEMOGRAMAN PYTHON BERBASIS KOMPUTERISASI. *INFOTECH
journal*, 9(1), 11–19. <https://doi.org/10.31949/infotech.v9i1.4233>

Prasetyo, S., & Dewayanto, T. (n.d.). *PENERAPAN MACHINE LEARNING, DEEP
LEARNING, DAN DATA MINING DALAM DETEKSI KECURANGAN
LAPORAN KEUANGAN - A SYSTEMATIC LITERATURE REVIEW.*

Rochmawati, N., Hidayati, H. B., Yamasari, Y., Tjahyaningtijas, H. P. A., Yustanti, W.,
& Prihanto, A. (2021). Analisa Learning Rate dan Batch Size pada Klasifikasi
Covid Menggunakan Deep Learning dengan Optimizer Adam. *Journal of
Information Engineering and Educational Technology*, 5(2), 44–48.
<https://doi.org/10.26740/jieet.v5n2.p44-48>

Septhya, D., Rahayu, K., Rabbani, S., Fitria, V., Rahmaddeni, R., Irawan, Y., &
Hayami, R. (2023). Implementasi Algoritma Decision Tree dan Support Vector
Machine untuk Klasifikasi Penyakit Kanker Paru: Implementation of Decision
Tree Algorithm and Support Vector Machine for Lung Cancer Classification.

MALCOM: Indonesian Journal of Machine Learning and Computer Science,
3(1), 15–19. <https://doi.org/10.57152/malcom.v3i1.591>

Yudianto, M. R. A., & Fatta, H. A. (2020). *ANALISIS PENGARUH TINGKAT
AKURASI KLASIFIKASI CITRA WAYANG DENGAN ALGORITMA
CONVOLUTIONAL NEURAL NETWORK.*